

TUS OS™ White Paper: Verifiable AI Decision Receipts and Domain Binding

Status

- This publication contains the *TUS OS White Paper v1* (dated 2026-03-04). The white paper is **informational** and describes concepts and intended design goals.
- **Normative requirements, conformance criteria, and certification rules are not yet defined in this release.** Those will be provided in future versioned publications (Protocol Specification, Canonicalization Rules, and Conformance Test Suite).

Artifact integrity

- File: `TUS OS™ White Paper – Verifiable AI Decision Receipts and Domain Binding.pdf`
- SHA-256 embedded in the PDF.

Roadmap

- Planned publications: Protocol Specification v1.0 (normative), Conformance Test Suite v1.0, Reference Implementation, Trademark & Compatibility Policy.

Trademark

- “TUS OS” is the trademark of `Mark Whitlock`. This publication does not grant any rights to use the trademark except as permitted by applicable law. A detailed trademark and compatibility policy will follow.

Mark Whitlock

4 March 2026

TUS in one paragraph (plain-language). TUS provides a missing primitive for accountable decision systems: it binds analysis and action to a **verifiable, immutable decision artefact** (a canonical receipt). Under the hood, TUS uses **deterministic decision geometry** over a small, enumerable state space to produce a one-step recommendation and the intermediates needed to replay it. It also defines a **binding language**—Domain Profiles and the Semantics Binding Layer—that bridges **structure (receipt fields)** and **meaning (domain claims)** without letting interpretation rewrite computation. In environments where trust, compliance, and dispute resolution dominate—finance, insurance, public-sector allocation, enterprise governance, and AI oversight—post-hoc explanations and opaque outputs are not enough; an independent party should be able to **recompute, validate, and audit** the decision from recorded inputs.

Executive summary

TUS (as specified in this publication and its referenced artefact bundles) is a **deterministic, geometry-first decision system** defined over a **finite 6-bit state space (64 states)**, coupled to a **receipt protocol** that makes each evaluation **recomputable, auditable, and content-addressable**.

Its core coherence mechanism is **determinism under conformance**: for a fixed input state, TUS computes a canonical one-step measurement record `s(h)` and a canonical distribution over the six permitted “micro-moves” (single line flips), then selects a deterministic transition in **GUIDANCE** mode (deterministic argmax selection; frozen tie-break). The system is explicitly designed to avoid **semantic backflow**: interpretation is downstream only and is never permitted to change computation.

Its excellence mechanism is **receipt-first accountability**: the primary artefact is a canonical JSON receipt that includes (a) the evaluated state, (b) neighbourhood alternatives, (c) computed intermediates (fragmentation field, entropy potential, softmax terms, rankings), (d) the chosen transition, and (e) integrity metadata (canonical hashing). This enables independent verification, regression testing (including full 64-state sweeps), and disciplined improvement without interpretive narratives drifting “by accident.”

Evidence in this publication and its referenced artefact bundles supports the **design and specification** of a fully specified kernel, a conformance concept (Tier A vs Tier B), a validator mindset, and an example “triad” bundle (GUIDANCE + LANDING receipts plus triad-level foundation/synthesis artefacts and a manifest). However, **external empirical validation of decision quality** (benchmarks against real-world outcomes, controlled trials, domain-specific datasets) is **not provided** in the referenced artefact bundles; performance claims are primarily about determinism, auditability, and verification properties.

Maturity posture. The kernel and receipt protocol are specified, with conformance/validator concepts and example artefacts demonstrating deterministic, replayable runs. Reference implementation repository is currently private while in pilot; verification artefacts (canonical receipts, audit pointer maps, manifests) are distributed as versioned bundles with SHA-256 identifiers. External outcome validation (benchmarks against real-world results, controlled trials) and evidence of production deployment in relevant operational environments are not yet published.

Definition and scope of TUS

TUS is defined (in the technical write-up) as a **deterministic decision framework** that treats a “cast” as selecting a discrete state in a **fully enumerable 64-state space** with a defined neighbourhood: from any state there are exactly six immediate alternatives reachable by flipping exactly one line/bit.

A key scope constraint is conceptual: TUS explicitly positions itself **against prophecy and inherited symbolism**; it treats the 64 states as a bounded combinatorial field and makes recommendations only as **local, one-step guidance** under a declared question scope, not as a claim about the future.

In the Semantics Binding Manual, TUS is also framed as a **protocol for turning a decision process into a tamper-evident, self-verifying record (“receipts”)**. This broadens “TUS” from merely a kernel to a **full stack**: kernel computation → canonical receipt emission → verification → controlled interpretation.

What is in scope by specification (v1/v1.1 as supplied):

- A fixed **6-bit** state model; deterministically defined neighbourhood; deterministic geometry computation; and deterministic GUIDANCE selection.
- A receipt-based integrity model (canonical JSON, canonical hashing projections, audits, conformance tiers) intended to support independent verification.
- A disciplined interpretation/binding model (SBL, Semantics Binding Layer, v1) that constrains narrative output to receipt-grounded claims, with explicit “no invented context” and “no prophecy” guardrails.

What is explicitly unspecified or outside the provided scope:

- Any claim that TUS improves real-world outcomes versus baselines (decision accuracy, business KPIs, reduced risk, etc.) via controlled evaluation.
- A universal, “one true” mapping from real-world situations to a cast/state selection that would apply across domains without a domain-specific contract.
- Any public governance model, licensing terms, or standards-body affiliation.

Domain Profiles and Domain Binding

TUS separates **computation** (the kernel) from **meaning** (domain semantics). The mechanism for binding a real-world workflow to a TUS run is the **Domain Profile**.

A Domain Profile is a **binding contract** that is recorded as part of the run and therefore becomes part of the decision artefact. It is not “interpretation” in the informal sense; it is a *declared interface* that fixes what the run is allowed to mean.

A Domain Profile typically specifies:

- **Scope and boundary rules:** what is in-bounds / out-of-bounds for the question; what counts as admissible input.
- **Roles and success criteria:** who the decision is for, what “success” means for this workflow, and which constraints are binding.
- **Domain term bindings:** how domain language maps onto TUS signals (distance, rank, pull) and/or specific receipt fields.
- **Evidence and claim grounding:** what claims are permitted in the narrative layer and which receipt paths they must cite (e.g., JSON-pointer grounding).

Avoiding semantic backflow

Domain Profiles are designed to prevent semantic backflow by enforcing a one-way direction:

- The **kernel** produces receipts from a cast/state under a frozen mathematical contract.
- The **Domain Profile** is bound into the receipt stack as an input declaration (versioned/hashed), so later readers can’t silently change scope, roles, or success criteria.
- The **narrative layer** is constrained by the profile’s grounding rules: claims must cite admissible receipt fields, so meaning cannot drift without a visible change in the bound profile.

This preserves the core guarantee: interpretation is downstream only, and any change in meaning must be expressed as a change in the declared, receipt-bound Domain Profile.

Architecture, components, algorithms, and data flows

TUS is architected as a layered system whose **kernel is strictly upstream** of all semantics. The technical write-up specifies that the kernel must not depend on user text, time, or external data after the cast, and that interpretation can only occur downstream from a verified receipt.

Kernel computation: state $\rightarrow$ local geometry $\rightarrow$ canonical distribution $\rightarrow$ selection

A state is a 6-bit vector $\mathbf{h} = (b_1 \dots b_6)$ with a frozen line indexing (bottom-to-top) and six neighbours $\mathbf{h}_i$ given by flipping line i .

Notation. In prose we write subscripts using Unicode (e.g., $\mathbf{h}_i$, $\mathbf{w}_i$, $\mathbf{q}_i$, $\mathbf{s}_i$). In the Mermaid diagram we use the ASCII-safe form `*-sub-i` (e.g., `h-sub-i`) to avoid renderer syntax issues. Bit values are mapped to spins $s_i \in \{-1, +1\}$ for the core primitive.

The only upstream primitive specified is a scalar fragmentation field $\text{frag}(\mathbf{h})$ defined as a weighted sum of pairwise disagreements between spins, with frozen weights w_{ij} determined by constants (α , β , small epsilons) and a distance-decay term.

From this primitive, the kernel computes:

- Neighbour contrast weights $w_i(\mathbf{h}) = |\text{frag}(\mathbf{h}_i) - \text{frag}(\mathbf{h})| + \epsilon$
- A local geometry distribution $q_i(\mathbf{h}) = w_i / \sum w$
- An entropy potential $\Phi(\mathbf{h}) = -\sum q_i \ln(q_i)$ with per-line contributions
- A centred and scaled vector derived from per-line entropy contributions, with a fixed scaling rule (no free temperature knob)
- A stabilized softmax (with frozen evaluation/summation order and tie-breaks) to produce the canonical move distribution $p_{\text{line}}(\mathbf{h})$ over the six line flips

GUIDANCE then selects $i^* = \text{argmax } p_{\text{line}, i}(\mathbf{h})$ (lowest-index tie-break) and transitions to $\mathbf{h}^* = \text{flip}(\mathbf{h}, i^*)$ (the neighbour obtained by flipping line i^*).

Policy modes and boundary rules

The specification separates three modes with hard boundaries:

- **CAST**: selects an initial state using the CAST rule: a **declared uniform stochastic selection** over the 64 states (documented in the receipt with replay evidence), with a frozen bit-to-line mapping.
- **GUIDANCE**: a deterministic evolution mode (geometry-descriptive, not causal) that selects the top-ranked move under a declared tie-break rule; receipts record intermediates so third parties can replay and verify the computation under the bound domain profile.
- **LANDING**: a **conditionally deterministic** evolution mode (geometry-descriptive, not causal): the initial anchor state $\mathbf{H}_0$ may be stochastic via CAST (recorded with replay evidence); **given** $\mathbf{H}_0$, all subsequent steps are deterministic under the declared `landing_rule` (with deterministic tie-breaks and lookahead, if enabled).

In the supplied triad bundle (as described in the Semantics Map), LANDING receipts include additional scaffolding fields (e.g., rank anchoring trace, two-step internal endpoints) to support scenario language while preventing prediction claims.

LANDING Determinism in LANDING_TRIAD_SET

Outside `LANDING_TRIAD_SET`, LANDING implementations may use stochastic line sampling; **within** `LANDING_TRIAD_SET`, **line selection is deterministic** (randomness, if used, is confined to the initial anchor state H_0).

LANDING is an evolution mode that makes **no predictive claims** by allowing a constrained non-deterministic degree of freedom: the **initial anchor state** (the first hex/state, H_0) may be selected under the declared CAST rule. **After H_0 , LANDING is fully deterministic**: subsequent selections are computed from the ranked neighborhood under a **declared deterministic landing_rule** (e.g., anchored-rank `RUNNER_UP` / `MEDIAN` / `LOWEST`), with deterministic tie-breaks and—where enabled—deterministic lookahead by reapplying the same anchor index across steps. In `LANDING_TRIAD_SET`, randomness is used **only** for the initial anchor selection; all derived starts (e.g., B,C) and all subsequent landing selections are deterministic, and the receipt records the `landing_rule` (and any RNG segment declaration for H_0) so an auditor can replay and verify the post-anchor computation exactly.

GUIDANCE Determinism and Verifiability

GUIDANCE is the system's deterministic evolution mode: given the same input state, domain binding, and kernel parameters, it produces the same selected move every time. The kernel evaluates the local neighborhood, computes the move distribution, and selects the top-ranked option via a declared deterministic rule (e.g., argmax with deterministic tie-break), producing a receipt that records enough intermediate structure to allow independent recomputation and validation. GUIDANCE makes no claim of “ground truth correctness”; it claims replayable computation under a fixed domain profile, ensuring that any narrative or interpretation remains downstream of—and bound to—the recorded decision trace.

Rank space, Elements, and the meaning interface

A central design move is transforming from line-indexed probabilities (`p_line`) to a **label-invariant rank space** (`p_rank`) using a deterministic sorting rule. This explicitly separates: (a) which line is which from (b) the shape of the distribution.

TUS defines Elements as a lossless coordinate chart over the rank distribution:

- `E1..E5` capture the leading mass and successive rank gaps (with reconstruction equations provided).
- `E6..E10` capture rank-space deltas when moving from `h` to `h*` under GUIDANCE.

The Semantics Map and the SBL manual treat these receipt fields as the admissible substrate for interpretation: narrative must be grounded in explicit JSON pointers to canonical receipt fields, and it must maintain rank-space vs line-space discipline to avoid a core foot-gun (mistaking “rank 1” for “line 1”).

Receipt and validation pipeline: computation → canonicalisation → hashing → audits → interpretation

Receipts are specified as the sole output artefact per query, sufficient to recompute and validate the one-step mapping and to audit any stochastic choices by replaying recorded inputs. The receipt schema includes normative blocks (meta/input/engine/state/selection/elements/audits/validity/integrity) and may include non-normative notes that validators ignore.

Numeric determinism is treated as a first-class requirement: floats are encoded in a dual representation `{v, s}` to preserve bit-exact round-tripping (including signed zero) and to prevent serializer drift from invalidating hashes.

Canonical hashing is defined over a projection (excluding volatile or non-normative fields), producing a content-derived identifier (SHA-256) that binds the receipt to its computation.

The recommended implementation layering explicitly keeps computation and canonicalisation out of UI code (the write-up recommends a Rust core with a thin wrapper UI), and introduces a validator plus a conformance runner (full 64-state sweep) to enforce Tier A.

Receipt schema at a glance (Tier A hashed view vs full receipt)

The canonical receipts in this slice expose **two layers**:

1. a **Tier A hashed view** (the `hash_projection` named `RECEIPT_VIEW_V1`), which is the object canonicalised and hashed to produce `integrity.canonical_hash`; and
2. a **full receipt envelope**, which may include additional blocks (e.g., `start`, `envelope`, `receipt_set`, and LANDING-only scaffolding) that are **permitted but excluded** from the Tier A canonical hash.

This distinction is not editorial: it is observable directly in the audit appendices, which provide an exhaustive JSON-Pointer map for the **hashed view** only.

Tier A minimum required blocks (hashed view: `RECEIPT_VIEW_V1`)

The Tier A hashed view consists of these **nine** top-level blocks (as enumerated in Appendix A pointer maps in the audits):

- `meta`
- `input`
- `engine`
- `state`
- `selection`
- `elements`
- `audits`
- `validity`
- `integrity`

Within these blocks, the validator expects (at minimum) the following *structural anchors* to exist (representative, not exhaustive):

- `meta`: `/meta/conformance_tier`, `/meta/created_utc`, `/meta/mode`, `/meta/receipt_schema/*`, `/meta/engine_spec/*`, `/meta/engine_build/*`, `/meta/determinism/*`
- `input`: `/input/origin`, `/input/query_context`, and state encodings such as `/input/state_bin`, `/input/state_hex`, `/input/state_id`
- `engine`: frozen parameters and contracts such as `/engine/alpha`, `/engine/beta`, `/engine/fp_contract`, `/engine/math_backend_id`, `/engine/neighbor_rule`,

`/engine/softmax/*`, `/engine/tie_break`, `/engine/toolchain_id`

- `state`: core computed fields (e.g., `/state/Phi*`, `/state/Phi_terms*`, `/state/T*`, `/state/frag_h*`, `/state/frag_neighbors*`, `/state/W*`) plus `/state/core_hash`
- `selection`: transition record (`/selection/from_state_hex`, `/selection/to_state_hex`, and the mode-specific choice fields)
- `elements`: rank-space chart and deltas (`/elements/E1..E10`, and the rank support arrays like `/elements/p_rank_h*`)
- `audits`: audit commitments (`/audits/branches/*`, `/audits/field_view/*`, `/audits/lookahead/*`)
- `validity`: gate results (`/validity/gates/*/pass`, plus diagnostics)
- `integrity`: hashing metadata (`/integrity/canonical_hash`, `/integrity/hash_alg`, `/integrity/hash_projection`, `/integrity/prev_receipt_hash`)

Mode-specific deltas inside the hashed view

Even within the hashed view, GUIDANCE vs LANDING differs in a small, explicit way in this slice:

- GUIDANCE includes `/selection/chosen_line` and `/selection/chosen_p_line`.
- LANDING replaces that with `/selection/anchored_line` and `/selection/landing_rule` and adds RNG segment declarations under `/audits/lookahead/*`.

Full receipt envelope (excluded from Tier A hash)

Outside the hashed view, the canonical JSON receipts in this slice also carry additional top-level blocks:

- Common: `start`, `envelope`, `receipt_set`.
- LANDING-only (triad scaffolding): `landing_anchor`, `triad_anchor_trace`, `rank_selected`, `total_ranks`, `chosen_line_step1`, `chosen_line_step2`, `landing_to_state_hex`, `landing_to_state_id`.

These are useful for packaging, provenance, and narrative scaffolding, but they are not part of the `RECEIPT_VIEW_V1` pointer map and therefore do not influence the Tier A canonical hash.

Normative source of truth for exact field sets

For a strict implementation, **Tier A validator conformance is defined by the Appendix A JSON-Pointer map** emitted by the audit renderer for `hash_projection = RECEIPT_VIEW_V1`. The whitepaper should not attempt to reprint the full pointer list inline (it runs to thousands of pointers), but can reference the audit artefacts as the authoritative index for validators.

Interpretation SHOULD ground narrative claims to stable receipt paths within the hashed view (for example: `selection.*`, `state.Phi*`, `elements.E2`, `integrity.canonical_hash`) under the bound Domain Profile.

flowchart TD

```
A["Declare question scope"] --> B["CAST selects state h"]
B --> C["Compute frag(h) and frag(h-sub-i)"]
C --> D["Compute W-sub-i and q-sub-i"]
D --> E["Compute Phi, Phi terms, centering, and T"]
E --> F["Softmax to get p-line(h)"]
F --> G["Rank transform to get p-rank and pi-h; elements E1 to E5"]
G --> H["Guidance: choose i-star = argmax p-line; set h-star"]
H --> I["Recompute rank at h-star; delta p-rank; elements E6 to E10"]
I --> J["Emit canonical receipt JSON and canonical hash"]
J --> K["Validator and audits"]
K --> L["Interpreter renders meaning (receipt-grounded)"]
```

The flow above mirrors the architecture-mirroring principle enforced by SBL (Semantics Binding Layer) v1.1 (State → Neighbourhood → Distribution → Rank → Transition → Lookahead → Coverage → Synthesis).

Threat model and non-goals (clarifying what receipts do and do not guarantee)

Receipts are designed to prevent **silent drift and post-hoc rationalisation**: they make the computation replayable, constrain evaluation order/serialization so hashes are stable, and bind scope/Domain Profile declarations so meaning cannot be altered without a visible change to the bound inputs.

Receipts do **not** guarantee *decision correctness* in the real world. They do not prevent (a) an ill-posed question scope, (b) a poor or adversarial Domain Profile binding, (c) “garbage-in/garbage-out” from an unrepresentative cast, or (d) downstream misuse of narrative beyond the profile’s grounding rules. Outcome quality therefore requires domain-specific evaluation and governance in addition to conformance/validation.

Intended and potential use cases, measurable claims, and empirical validation

Intended use cases and domains

The Semantics Binding Manual explicitly motivates TUS as an accountability primitive for environments where audit, compliance, and dispute resolution dominate, naming domains such as **finance, insurance, public-sector allocation, enterprise governance, and AI oversight**, and positioning receipts as a way to move from “because the model said so” to a verifiable derivation from declared rules.

Within the triad framework described in the Semantics Map, the system also supports multi-region exploration: multiple starting states (anchor + derived) are scanned with paired GUIDANCE and LANDING receipts, then compared under synthesis guardrails to prevent over-interpretation.

Measurable performance claims present in the artefacts

The referenced artefact bundles support verification-centric performance claims, rather than outcome-centric claims:

- **Determinism under conformance**: Tier A is defined as bitwise identical outputs/receipts under a strict floating-point and evaluation-order contract, supported by a notion of a frozen math backend / corpus and canonical serialization rules.

- **Auditability / recomputability:** receipts are designed to contain sufficient intermediates to recompute and validate a one-step mapping and to audit stochastic choices (when present).
- **Enumerability and exhaustive diagnostics:** because the space is 64 states with 6 neighbours each, diagnostics can be exhaustive (at least for v1), which is a strong property for validator completeness and regression testing.

Empirical validation, benchmarks, tests, and datasets

What exists in the referenced artefact bundles (validation evidence):

- A concept of an official/canonical vector corpus and conformance runner sweeping all 64 states to validate Tier A behaviour.
- A triad bundle (distributed as a versioned artefact bundle) containing canonical receipts and triad artefacts; the Semantics Map documents the internal file set and the stable semantic substrate fields expected across receipts.

What is unspecified (benchmarks/outcome validation):

- Any benchmark dataset linking receipt metrics (Φ , `p_line`, Elements, etc.) to external ground truth, expert judgement, or decision outcomes.
- Any controlled comparison to alternative decision aids in terms of decision quality, calibration, or user performance.
- Any published performance measurements for throughput/latency, receipt size distributions, or verification cost across platforms.

Qualitative example (receipt-grounded, non-benchmark)

A useful way to understand TUS without implying outcome validation is to look at the shape of a receipt and the kinds of statements it permits.

A triad run produces three receipt-backed reports (one per anchor). Each report includes the canonical move distribution (`p_line`), its rank-transformed shape (`p_rank`), and the Elements (`E1..E10`) that encode the distribution and its one-step change.

This supports qualitative, checkable claims such as:

- **Near tie at the top:** when the top gap in rank space (e.g., `E2`, the separation between rank 1 and rank 2) is small, the receipt supports the statement that two moves are closely competing.
- **Dominant move:** when the leading mass and gap structure indicate clear separation, the receipt supports the statement that one move dominates under the declared mode.
- **Angle sensitivity:** if the same qualitative pattern (near tie vs dominance) appears across all three anchors, it suggests a stable feature of the framing; if it differs by anchor, it flags that starting assumptions are doing more work.

These examples illustrate the intended posture: the system makes verification-centric claims about what the computation shows under a declared scope, not empirical claims about real-world decision quality.

Value proposition, novelty, and comparison to comparable systems

Value proposition and stakeholders

TUS's value proposition is strongest where the primary requirement is **coherence under scrutiny**: the ability to show, replay, and verify what was evaluated and why a particular transition was chosen. This directly benefits:

- Decision owners who need repeatable, defensible reasoning trails.
- Auditors, governance bodies, and compliance stakeholders who require inspectable artefacts instead of post-hoc rationales.
- AI/ML teams that need a deterministic commit layer around probabilistic exploration (TUS framed as a governance wrapper).

Novelty: what appears new vs derivative

Materially novel contributions (as a combined system):

- The receipt-as-product principle: a canonical receipt is the primary deliverable, not a narrative or a score.
- Bit-level determinism mechanisms in a decision context (dual float `{v,s}`, canonical hashing projections, explicit evaluation order) aimed at preventing drift by implementation detail.
- A two-layer semantics governance framework (Semantics Map + SBL manual) that treats interpretation as a constrained compilation from receipt metrics with explicit prohibitions (no prophecy, no invented context, coordinate safety).
- The rank-space Elements chart that is designed to be lossless for the rank distribution while enabling controlled human-meaning bindings.

Derivative components (well-known building blocks):

- Shannon entropy and softmax as generic tools for compressing distributions and deriving preference mass from centred signals.
- Canonicalisation + hashing patterns analogous to standard JSON canonicalisation schemes used to make JSON hashable for cryptographic operations (e.g., the JSON Canonicalization Scheme, RFC 8785).
- Provenance/attestation motivations similar to software supply chain provenance systems (e.g., in-toto; SLSA provenance).

Comparison to comparable systems and standards

Analytical takeaway. TUS sits at an intersection: it resembles provenance/attestation systems in its emphasis on verifiable artefacts, but unlike PROV/in-toto/SLSA it also defines a **native decision geometry kernel** and an **interpretation governance layer**. This makes it potentially powerful as a decision-receipt primitive, while also making it harder to standardise without clarifying scope (decision kernel vs general receipt protocol).

Appendices

The sections below are **non-normative** supporting material. They are included to help situate TUS relative to adjacent standards and to outline possible future interoperability directions. They do not claim these integrations are implemented in v1.

Informal comparison table (non-normative)

The following table is provided as a non-normative, informal comparison to help situate TUS relative to adjacent provenance/attestation/policy artefact systems. It is not intended as a definitive equivalence claim.

System / standard	Core purpose	Architecture / artefact model	Maturity evidence	“Performance” evidence (what is measured)	Primary use cases	Openness
TUS (as specified here)	Deterministic local decision geometry + verifiable receipts	Fixed 64-state kernel; canonical receipt JSON with hashes, audits, rank metrics, and interpretation governance; GUIDANCE vs LANDING separation.	Specified kernel; conformance tiers; triad example bundle; concept of full 64-state conformance sweep.	Verifiability, determinism, recomputability; no outcome benchmarks supplied.	Auditable decision records; governance wrappers around AI; structured scenario exploration with anti-prophecy guardrails.	Pilot / private repo (reference implementation private; verification artefacts distributed as versioned bundles with SHA-256 identifiers).
PROV (W3C PROV-DM)	Interchange model for provenance	Graph model of entities/activities/agents and relations for provenance interchange.	Published W3C Recommendation (stable, widely cited).	Expressiveness for provenance representation; not a decision kernel.	Data lineage, accountability, provenance capture across systems.	Open W3C standard.
in-toto	Secure integrity of software supply chains	Signed metadata (links/layouts) describing steps; verification ensures expected steps and authorised actors.	Academic paper (USENIX Security 2019) + maintained docs; used in supply chain tooling.	Integrity guarantees for pipeline steps; verification of signed metadata and rules.	Build/test/package pipelines; tamper resistance; compliance evidence.	Open-source project with published specs/docs.
SLSA (spec v1.2)	Maturity framework for supply chain security + provenance requirements	Levels/tracks; recommends attestation formats including provenance; focuses on preventing tampering and improving build integrity.	Published living specification (v1.2) with growing ecosystem adoption.	Assurance level classification; provenance requirements; not a decision kernel.	Supply chain governance, build security, provenance compliance.	Open specification.
Open Policy Agent	Policy decision engine for distributed services	Policy-as-code; decisions can be logged (decision logs) for auditing and observability.	CNCF-graduated project with production usage patterns.	Operational performance and correctness of policy evaluation; auditable decision logs.	Authorisation/guardrails; compliance logging; policy enforcement.	Open-source (Apache-style governance).
Model Cards (paper/framework)	Standardised documentation for ML models	Documentation artefact describing intended use, evaluation, caveats; focuses on transparency and benchmarking disclosure.	Highly cited peer-reviewed/archival publication; adopted as practice.	Empirical evaluation disclosure and reporting; not deterministic execution receipts.	Responsible ML release; stakeholder communication; risk mitigation.	Open academic proposal (implementation varies).

Possible interoperability and standardisation pathways (non-normative, not yet implemented)

TUS receipts are JSON-based and include canonicalisation/hashing rules; this is conceptually aligned (not equivalent) with the goal of RFC 8785: making JSON deterministically serialisable for cryptographic hashing/signing.

A realistic interoperability direction is to treat a TUS receipt as an attestable object:

- **Signing/packaging** could use JOSE/JWS for JSON signatures (or a COSE representation if a binary format is desired), because both are established standards for integrity protection over structured content.
- **Transparency logging** could be achieved by recording receipt hashes (or signed receipts) in an append-only transparency log such as Rekor (Sigstore), providing public verifiability of inclusion and tamper evidence.
- **Provenance graph integration** could map decisions/receipts into W3C PROV constructs (e.g., a receipt as an Entity generated by an Activity with an Agent), enabling integration with data lineage tooling.
- **Supply chain security alignment** could embed receipt hashes or decision receipts into in-toto/SLSA-style attestations if TUS is used as a governed decision gate in CI/CD or deployment approvals.

These are compatibility pathways; the supplied TUS documents do not claim such integrations exist today.